

SEQUENCES IN PYTHON

Objective:

The objective of this lab is to introduce students to discrete time sequences and basic operations on these sequences. Specifically, this lab will cover how to plot discrete time sequences using Python.

Equipment Needed:

Python 3.x

NumPy library

Matplotlib library

Introduction

A **discrete signal** is denoted by $x[n]$, in which the variable n is an integer value. In this lab we will learn to represent sequences in Python. The Independent variable (n) is assumed to represent time.

Discrete Time Signals/Sequences

In signal processing, discrete-time sequences are signals that are defined for discrete value of time independent variable n . These signals can be represented mathematically as functions of time.

In Python, Discrete-time sequences can be generated using the **NumPy** library and plotted using the **Matplotlib** library, **Pyplot** class: built-in function **Stem(n, x)**. Here are some examples of discrete-time signals and their Python code:

Unit Step Sequence

$$u[n] = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases}$$

The unit step function, denoted by $u[n]$. It is defined such that it is zero for $n < 0$ and for all the positive values of n including $n = 0$, its value is one.

Unit Impulse Signal

$$\delta[n] = \begin{cases} 1, & n = 0 \end{cases}$$

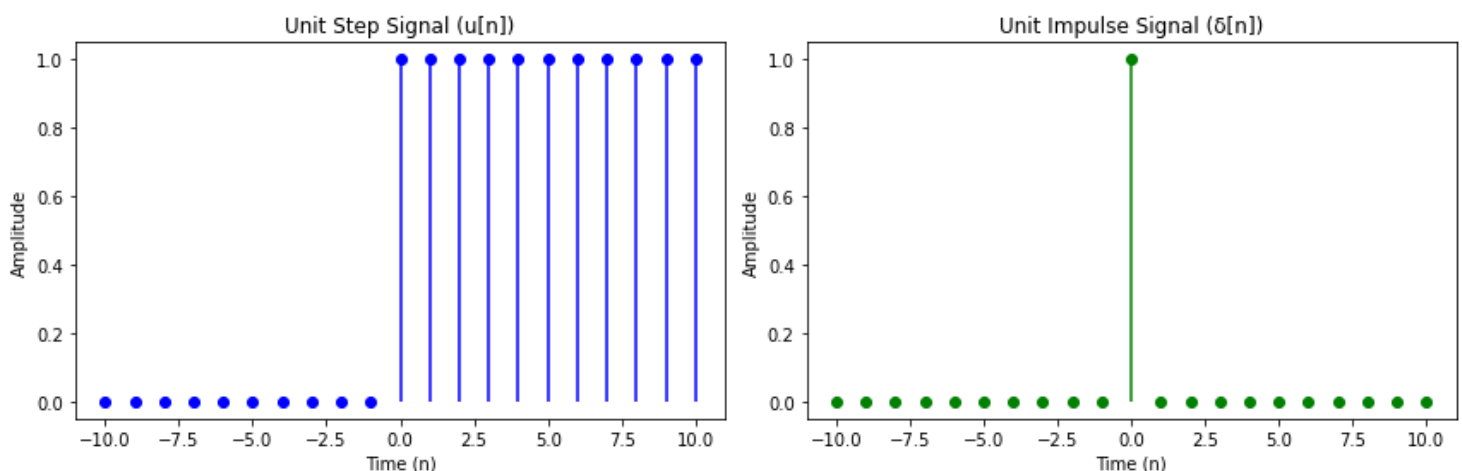
$$0, \quad n \neq 0$$

The unit impulse function, denoted by $\delta[n]$, is also known as the Dirac delta function. It is defined such that it is zero everywhere except at $n = 0$, where its value is one.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Define time axis (discrete)
n = np.arange(-10, 11)
# Unit Step Signal
u = np.where(n >= 0, 1, 0)
# Unit Impulse Signal
delta = np.where(n == 0, 1, 0)
# Plotting
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.stem(n, u, 'b', markerfmt='bo', basefmt=" ", use_line_collection=True)
plt.title('Unit Step Signal (u[n])')
plt.xlabel('Time (n)')
plt.ylabel('Amplitude')
plt.subplot(1, 2, 2)
plt.stem(n, delta, 'g', markerfmt='go', basefmt=" ", use_line_collection=True)
plt.title('Unit Impulse Signal (δ[n])')
plt.xlabel('Time (n)')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
```



Unit Ramp Signal

The unit ramp function is also used to define other functions, such as the step function and the impulse function. The step function is defined as the derivative of the ramp function, while the impulse function is defined as the second derivative of the ramp function. The unit ramp function starts at 0 for negative values of n and increases linearly with time for $n \geq 0$.

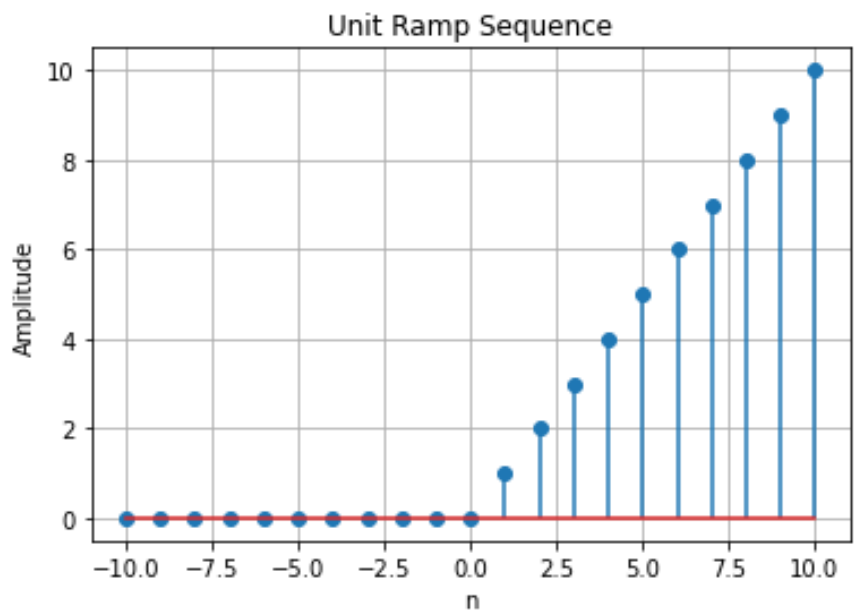
$$r[n] = \begin{cases} n, & n > 0 \\ 0, & n \leq 0 \end{cases}$$

It is commonly used to represent a signal that starts from zero and increases linearly with time, such as the position of an object that is moving with a constant velocity.

In Python, we can define and plot the unit ramp function using the following code:

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
# Generate sequence of n values
n = np.arange(-10, 11)
print(n)
# Generate unit ramp sequence
ramp = np.maximum(0, n)
print(ramp)
# Plot the sequence
plt.stem(n, ramp)
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('Unit Ramp Sequence')
plt.grid(True)
plt.show()
```



This Python code generates and plots the unit ramp sequence using NumPy and Matplotlib:

We first import the necessary libraries numpy and matplotlib.pyplot.

We generate a sequence of 21 values for n using the `np.arange()` function. This sequence ranges from -10 to 10.

We print the sequence of n values to the console for verification.

We generate the unit ramp sequence using the NumPy `np.maximum()` function. This function takes two arrays as input and returns an array with the maximum values of each corresponding element. Here, we pass 0 as the first argument and the array of n values as the second argument. This ensures that the ramp starts at 0 for negative values of n .

We print the resulting sequence of ramp values to the console for verification.

We use the Matplotlib `plt.stem()` function to plot the sequence. The first argument to this function is the sequence of n values, and the second argument is the sequence of ramp values.

The `stem()` function plots discrete values using a vertical line and a marker, which is suitable for discrete signals such as the unit ramp sequence.

We set the x-axis label to ' n ', the y-axis label to 'Amplitude', and the plot title to 'Unit Ramp Sequence'. We also turn on the grid lines using the `grid(True)` function.

Finally, we use the `show()` function to display the plot.

When you run this code, you should see a plot of the unit ramp sequence that starts at 0 for negative values of n and increases linearly with time for $n \geq 0$. The x-axis represents the index n , and the y-axis represents the amplitude of the signal. The plot displays discrete values using the `stem()` function instead of the continuous line plot used in the continuous case.

Exponential Sequences

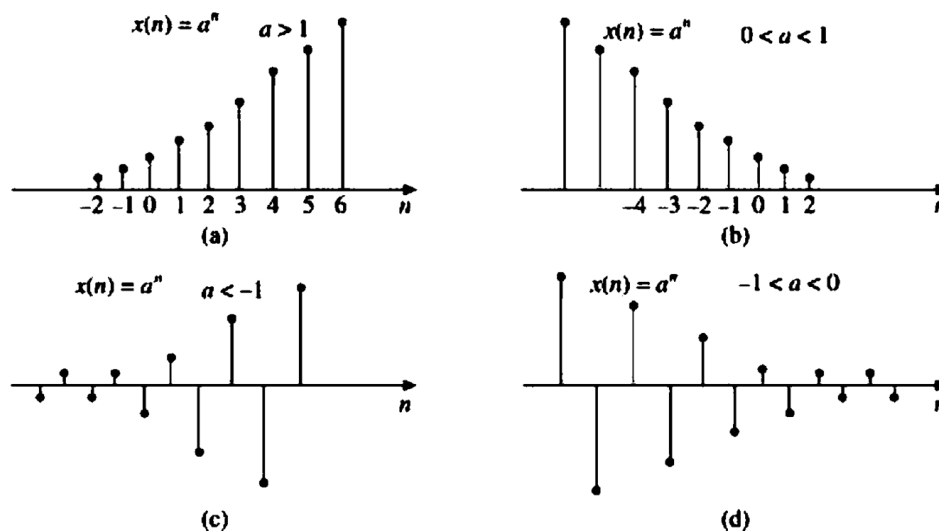
$$x(n) = a^n \quad \text{for all } n$$

Figure 1.8 illustrates different types of discrete-time exponential signals.

When $a > 1$, the sequence grows exponentially as shown in Figure 1.8(a).

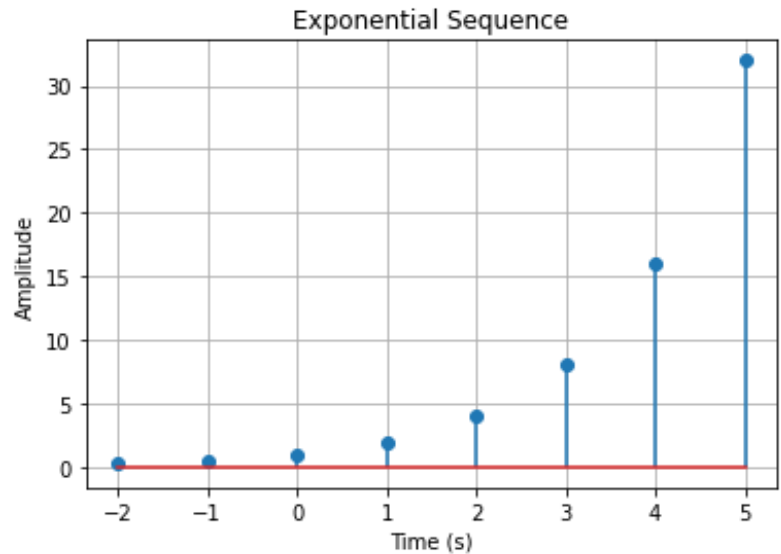
When $0 < a < 1$, the sequence decays exponentially as shown in Figure 1.8(b).

When $a < 0$, the sequence takes alternating signs as shown in Figure 1.8(c) and (d).



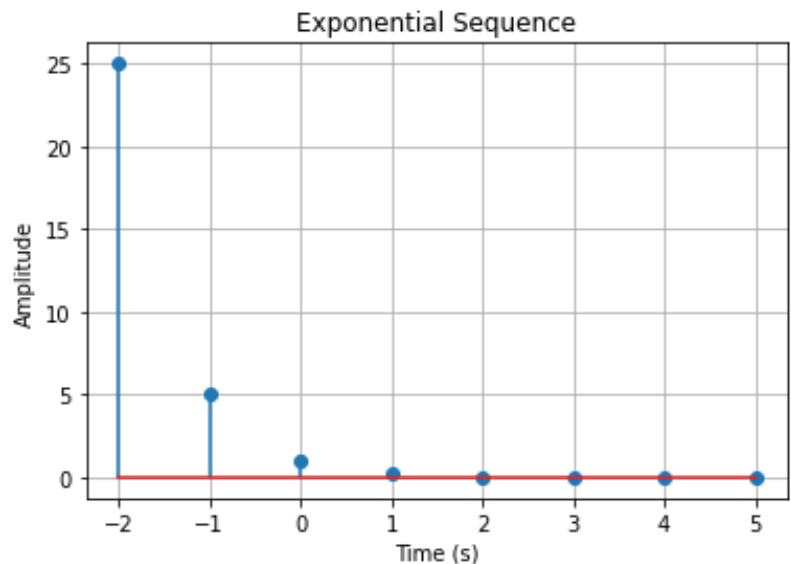
#####Increasing Exponential Sequence

```
import numpy as np
import matplotlib.pyplot as plt
# Define the parameters of the exponential
signal
a = 2 # constant
# Generate the time vector
n = np.arange(-2, 6, 1.0)
# Generate the exponential signal
x = a**n
# Plot the signal
plt.stem(n, x)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Exponential Sequence')
plt.grid(True)
plt.show()
```



#####Decreasing Exponential Sequence

```
import numpy as np
import matplotlib.pyplot as plt
# Define the parameters of the exponential
signal
a = 0.2 # constant
# Generate the time vector
n = np.arange(-2, 6, 1.0)
# Generate the exponential signal
x = a**n
# Plot the signal
plt.stem(n, x)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Exponential Sequence')
plt.grid(True)
plt.show()
```



Basic Signal Operations

Basic signal operations for discrete-time sequences involve similar concepts to continuous-time signals, but with discrete indices instead of continuous time. Here are some fundamental operations with examples:

1. Addition/Subtraction:

- **Operation:** Adding or subtracting two sequences element-wise.

- **Example:**
 - $y[n] = x_1[n] + x_2[n]$, where $y[n]$ is the sum of sequences $x_1[n]$ and $x_2[n]$.
 - $z[n] = x_1[n] - x_2[n]$, where $z[n]$ is the difference between sequences $x_1[n]$ and $x_2[n]$.
- **Explanation:** Like continuous-time signals, addition combines signals while subtraction isolates $y[n] = A \cdot x[n]$, where $y[n]$ is the scaled version of $x[n]$ by a constant A . components. For example, adding two discrete-time sequences would result in a new sequence where each sample is the sum of corresponding samples from the original sequences.

2. Scaling:

- **Operation:** Multiplying a sequence by a constant.
- **Example:**
- **Explanation:** Scaling changes the amplitude of the sequence without altering its shape or frequency content. For example, increasing the amplitude of a digital audio signal makes it louder.

3. Time Shifting:

- **Operation:** Delaying or advancing a sequence in time.
- **Example:**
 - $y[n] = x[n - k]$, where $y[n]$ is the delayed version of $x[n]$ by k samples.
 - $z[n] = x[n + k]$, where $z[n]$ is the advanced version of $x[n]$ by k samples.

$$y[n] = x[-n], \text{ where } y[n] \text{ is the time-reversed version of } x[n].$$

- **Explanation:** Time shifting moves the entire sequence along the time axis without changing its shape or amplitude. For example, delaying a digital audio sequence causes it to start playing later in time.

4. Time Reversal:

- **Operation:** Reversing the order of samples in a sequence.
- **Example:**
- **Explanation:** Time reversal flips the sequence around a central point. For example, a time-reversed digital audio sequence would play backward.

5. Time Scaling (Compression/Expansion):

- **Operation:** Stretching or compressing a sequence in time.

- **Example:**
 - $y[n] = x[an]$, where $y[n]$ is the time-compressed version of $x[n]$ by a factor $a > 1$.
 - $z[n] = x[bm]$, where $z[n]$ is the time-expanded version of $x[n]$ by a factor $0 < b < 1$.
- **Explanation:** Time scaling alters the duration of the sequence without changing its shape. For example, compressing a digital audio sequence makes it play faster, while expanding it makes it play slower.

Python Code for Basic Operations:

```
import numpy as np
import matplotlib.pyplot as plt

# Function to generate unit impulse sequence
def unit_impulse(n):
    return np.where(n == 0, 1, 0)

# Function to generate unit step sequence
def unit_step(n):
    return np.where(n >= 0, 1, 0)

# Function to generate unit ramp sequence
def unit_ramp(n):
    return np.where(n >= 0, n, 0)

# Function to generate exponential sequence
def exponential(n, A, a):
    return A * np.exp(a * n)

# Sequence generation
n = np.arange(-10, 11) # Define the range of n
A = 1                  # Amplitude
a = 0.1                # Exponential growth/decay rate

# Original sequences
impulse = unit_impulse(n)
step = unit_step(n)
ramp = unit_ramp(n)
exponential_seq = exponential(n, A, a)

# Sequence operations
# Scaling
scaled_step = 2 * step
scaled_ramp = 0.5 * ramp
scaled_exponential = 0.5 * exponential_seq
```

```
# Time shifting
shifted_step = unit_step(n - 3)
shifted_ramp = unit_ramp(n - 3)
shifted_exponential = exponential(n - 3, A, a)

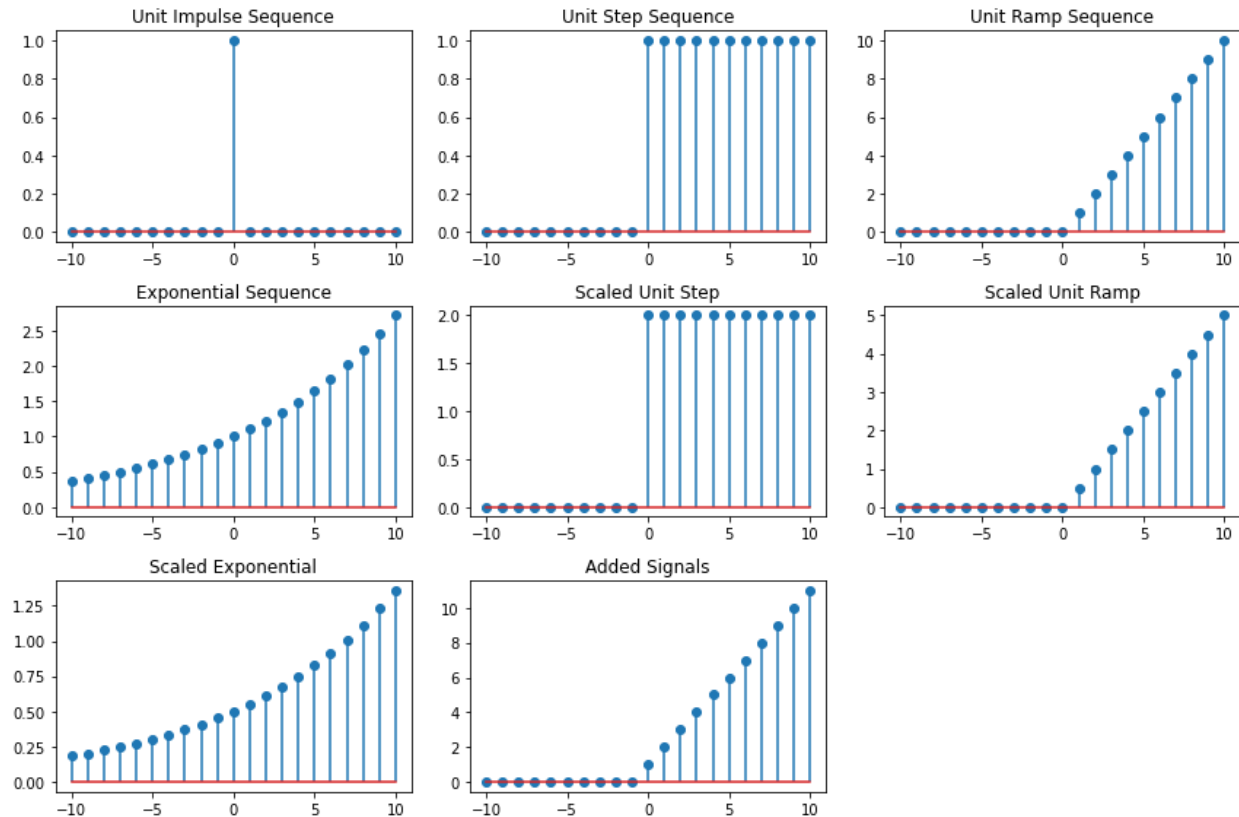
# Addition
added_signals = step + ramp

# Plotting
plt.figure(figsize=(12, 8))

# Original signals
plt.subplot(3, 3, 1)
plt.stem(n, impulse, use_line_collection=True)
plt.title('Unit Impulse Sequence')
plt.subplot(3, 3, 2)
plt.stem(n, step, use_line_collection=True)
plt.title('Unit Step Sequence')
plt.subplot(3, 3, 3)
plt.stem(n, ramp, use_line_collection=True)
plt.title('Unit Ramp Sequence')
plt.subplot(3, 3, 4)
plt.stem(n, exponential_seq, use_line_collection=True)
plt.title('Exponential Sequence')

# Modified signals
plt.subplot(3, 3, 5)
plt.stem(n, scaled_step, use_line_collection=True)
plt.title('Scaled Unit Step')
plt.subplot(3, 3, 6)
plt.stem(n, scaled_ramp, use_line_collection=True)
plt.title('Scaled Unit Ramp')
plt.subplot(3, 3, 7)
plt.stem(n, scaled_exponential, use_line_collection=True)
plt.title('Scaled Exponential')
plt.subplot(3, 3, 8)
plt.stem(n, added_signals, use_line_collection=True)
plt.title('Added Signals')

plt.tight_layout()
plt.show()
```

Lab Tasks

1. Run all above lab examples and experiment by changing time interval range, step values.
2. Write python scripts for all possible cases in discrete exponential sequences.
3. Apply all Signal Operations on above all explained sequences.
4. Plot $y[n] = 4*\delta[n+1] - \delta[n] + 2*\delta[n-1] + 6*\delta[n-2]$
5. Plot $z[n] = 4*\delta[n+2] + 2*\delta[n] - 3*\delta[n+2]$